

IBM CASE STUDY

Cybersecurity Network Threat & Intrusion Profiler

Name	V S Sravan Malladi
Roll No	A24126552268
IBM ID	IBMQ2DST2470
Department	CSE (AI & ML)
College Name	Anil Neerukonda Institute of Technology and Sciences
Module Name	Artificial Intelligence and Machine Learning
Module Number	6
UG Level	2

Declaration: This case study report and the accompanying prototype represent my own individual work, completed in accordance with the IBMQ2D Project Competition guidelines.

Table of Contents

S.No	Topic	Page No.
1	Case Study Opted	3
2	Overview	3
3	Business / Security Problem	3
4	Dataset	3
5	KPIs / Evaluation Metrics Defined	4
6	System & Model Design	4
7	Key Insights Surfaced	6
8	Security Impact and Recommendations	12
9	Challenges and Solutions	13
10	Outcome of Case Study	13
11	Conclusion	14
12	How to Run the Prototype	14

1. Case Study Opted

Case study title: Cybersecurity Network Threat & Intrusion Profiler (IBMQ2D Case Study #18, UG Level 2)

Modern computer networks face both known cyberattacks and newly invented attack methods. In this project, a machine learning system is built that uses classification models to catch known network attacks and anomaly detection algorithms to flag unusual, zero-day network traffic, using the NSL-KDD Network Intrusion Dataset.

- Tools used: Python, scikit-learn, XGBoost, SHAP, matplotlib/seaborn
- Dataset: NSL-KDD Network Intrusion Dataset (KDDTrain+ / KDDTest+)
- Deliverable: A working, modular ML prototype (supervised + unsupervised + hybrid pipeline) plus this case study report

2. Overview

Security teams cannot rely on a single detection strategy. A classifier trained on known attacks will confidently mislabel a genuinely new attack technique as normal traffic, simply because it has never seen anything like it before. Conversely, a purely anomaly-based system can flag unusual behaviour but cannot say what kind of attack it is dealing with, which slows down response.

This case study documents the design and build of a hybrid intrusion detection prototype that answers three questions at once: is this connection an attack, which known category does it belong to if so, and — critically — should this be treated as a possible zero-day threat that a human analyst needs to look at? All three questions are answered using the NSL-KDD dataset, the standard academic benchmark for network intrusion detection research.

3. Business / Security Problem

Security Operations Centres (SOCs) typically face three connected problems when relying on machine learning-based intrusion detection:

- Known-attack blind spots — classifiers trained on historical attack data cannot recognize attack families they were never shown, and will often default to labeling them as Normal traffic with high confidence (demonstrated directly in Section 7).
- Alert fatigue from naive anomaly detection — a standalone anomaly detector with no context produces high false-positive rates if not calibrated carefully, wasting analyst time.
- No clear escalation path — most academic intrusion-detection projects report a single accuracy number and stop, without offering a mechanism to separate 'confidently known attack' from 'genuinely needs human review.'

Goal: design a system that behaves the way a careful analyst would — trust a confident, corroborated known-attack call, but escalate anything ambiguous or statistically unusual instead of silently guessing.

4. Dataset

Name: NSL-KDD Network Intrusion Dataset — the de-duplicated, rebalanced successor to the original KDD Cup 1999 dataset, and the standard benchmark for network intrusion detection research.

Files used: KDDTrain+.txt (125,973 rows) and KDDTest+.txt (22,544 rows), each with 41 features plus a label.

NSL-KDD was used instead of the raw KDD Cup 1999 dataset because it removes duplicate records that make the original dataset artificially easy — evaluating on NSL-KDD's official train/test split produces realistic, non-inflated accuracy numbers.

Critically, KDDTest+ contains 17 attack types that never appear anywhere in KDDTrain+ at all (apache2, httptunnel, mailbomb, mscan, named, processtable, ps, saint, sendmail, snmpgetattack, snmpguess, sqlattack, udpstorm, worm, xlock, xsnoop, xterm). This was verified programmatically by comparing the sets of attack labels in both files, not assumed. These 3,750 rows form the true zero-day evaluation set used throughout Section 7.

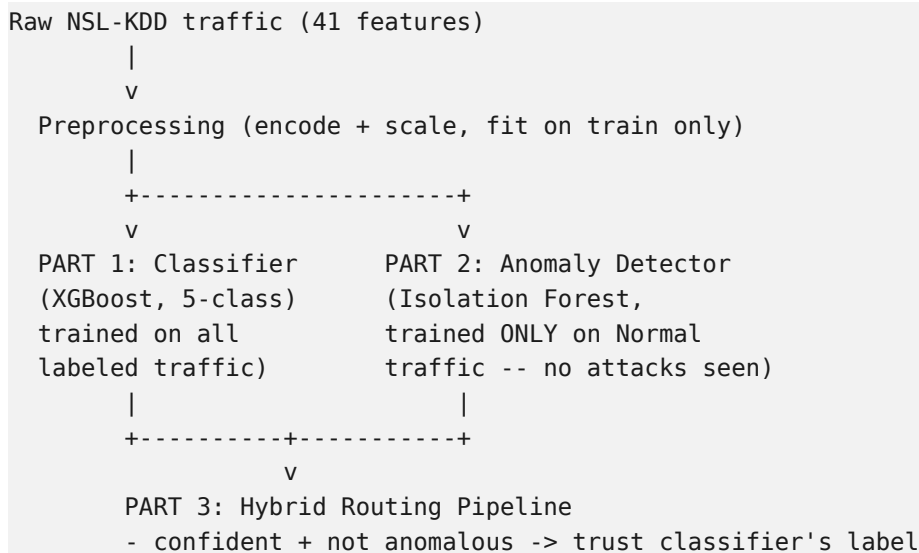
5. KPIs / Evaluation Metrics Defined

Before building any model, the metrics that would actually matter for this problem were defined up front — accuracy alone was ruled out early, since NSL-KDD is heavily imbalanced and a model can score 99% accuracy while missing nearly every rare attack.

KPI	Why it was chosen
Macro-averaged Precision/Recall/F1	Treats every attack class equally regardless of size — exposes rare-class failures that overall accuracy hides
Per-class recall (confusion matrix)	Shows exactly which attack types are being missed, and what they're being confused with
ROC-AUC / Average Precision (anomaly detectors)	Threshold-independent measure of how well an anomaly score separates Normal from Attack
True zero-day detection rate	Detection rate measured specifically on the 17 attack types absent from training — the real test of zero-day capability
Hybrid routing accuracy + false-alarm rate	Measures whether the combined system correctly escalates uncertain traffic without over-flagging normal traffic

6. System & Model Design

6.1 Architecture



- unconfident OR anomaly disagrees -> 'Unknown/Zero-Day Suspect'

6.2 Data Preprocessing

- Categorical features (protocol_type, service, flag) — one-hot encoded, since these are nominal categories with no natural order. handle_unknown='ignore' ensures unseen service values in KDDTest+ don't crash the pipeline.
- Numeric features (38 columns) — StandardScaler, since several downstream models (Local Outlier Factor, PCA/autoencoder) are scale-sensitive.
- Class imbalance — handled via class_weight='balanced' / sample-weighting rather than SMOTE, because U2R has only 52 training rows across a 122-dimensional encoded space; synthetic interpolation at that scale risks fabricating unrealistic 'attacks.'
- No data leakage — all preprocessing is fit exclusively on KDDTrain+; KDDTest+ only ever passes through .transform().

6.3 Model Design — Part 1: Supervised Classification

Three models were trained and compared on both a binary task (Normal vs Attack) and a multiclass task (Normal/DoS/Probe/R2L/U2R): Decision Tree, Random Forest, and XGBoost.

6.4 Model Design — Part 2: Unsupervised Anomaly Detection

Three anomaly detectors were trained using only the 67,343 Normal-labeled rows from KDDTrain+ — no attack signatures of any kind were shown during training, directly simulating the zero-day scenario. Isolation Forest, Local Outlier Factor, and a PCA-reconstruction-error detector (with an optional real autoencoder if TensorFlow is available) were compared.

6.5 Model Design — Part 3: Hybrid Combined Pipeline

The best classifier (XGBoost) and the best anomaly detector (Isolation Forest) are combined using an explicit decision rule: if the classifier is confident (predicted-class probability ≥ 0.60) and the anomaly detector doesn't disagree, the classifier's label is trusted; otherwise, the connection is routed to an 'Unknown/Zero-Day Suspect' bucket. The 0.60 threshold was chosen empirically from where correct-prediction confidence clusters on validation data.

6.6 Explainability Design

A SHAP (SHapley Additive exPlanations) analysis layer sits on top of the Part 1 classifier using shap.TreeExplainer, which computes exact SHAP values for tree ensembles. This produces global feature-importance rankings, per-class impact plots, and individual case-level explanations — directly answering 'why did the model decide this?' for any given connection.

6.7 Tools & Technology Stack

Component	Tool / Library
Language	Python 3.12
Data handling	pandas, numpy
Classical ML models	scikit-learn (Decision Tree, Random Forest, Isolation Forest, Local Outlier Factor, PCA)

Gradient boosting	XGBoost
Imbalance handling	class_weight balancing; imbalanced-learn available for SMOTE ablation
Explainability	SHAP (TreeExplainer)
Visualization	matplotlib, seaborn
Optional deep learning	TensorFlow/Keras (autoencoder, with automatic PCA fallback)

7. Key Insights Surfaced

7.1 Part 1 — Supervised Classification Results

Task	Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Binary	Decision Tree	0.778	0.815	0.801	0.777
Binary	Random Forest	0.779	0.817	0.802	0.778
Binary	XGBoost (best)	0.797	0.828	0.818	0.797
Multiclass	Decision Tree	0.748	0.583	0.503	0.490
Multiclass	Random Forest	0.739	0.765	0.475	0.479
Multiclass	XGBoost (best)	0.779	0.825	0.557	0.581

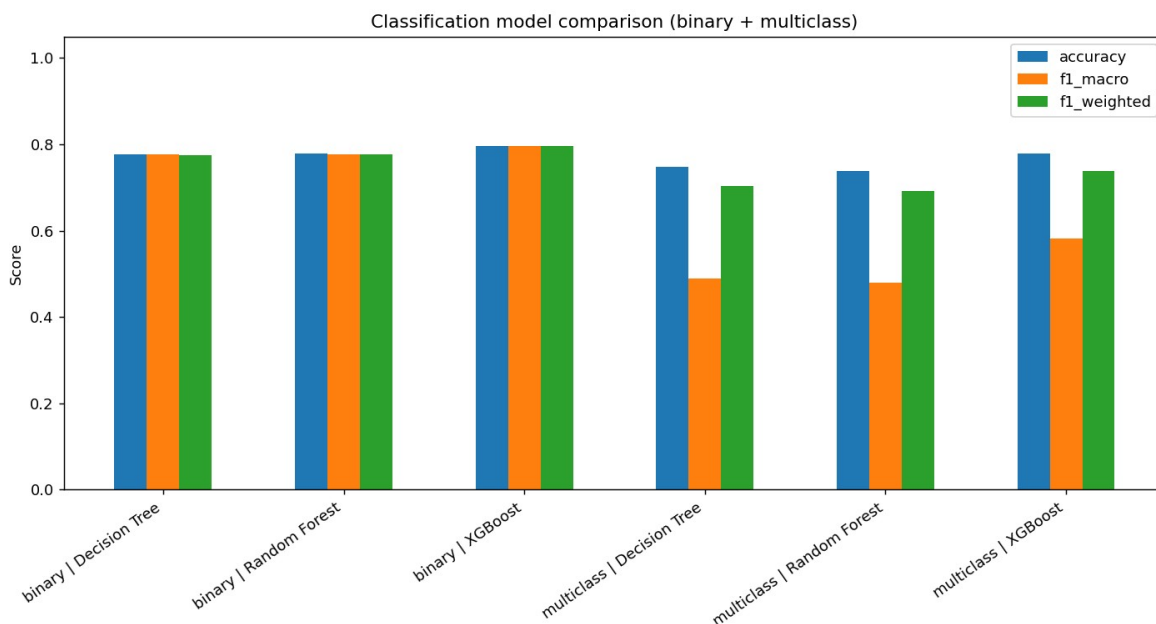


Figure 1 — Accuracy / macro-F1 / weighted-F1 across all six trained models.

XGBoost was the strongest model on every metric in both tasks, consistent with its known strength on structured/tabular data through iterative error correction rather than simple averaging (Random Forest) or a single overfit tree (Decision Tree).



Figure 2 — Multiclass confusion matrix for XGBoost: raw counts (left) and row-normalized per-class recall (right).

Normal (0.84) and DoS (0.97) are detected reliably, but R2L recall collapses to just 0.06 — 93% of true R2L attacks are misclassified as DoS. This is a direct consequence of R2L having only 995 training examples (0.8% of the training set) and most of its KDDTest+ variants belonging to attack families the model never saw during training.

7.2 Part 2 — Anomaly Detection Results (Zero-Day Simulation)

Model	ROC-AUC	Avg. Precision	Detection — all attacks	Detection — TRUE zero-day only
Isolation Forest (best)	0.942	0.954	67.3%	63.1%
Local Outlier Factor	0.881	0.857	22.5%	17.4%
PCA Reconstruction Error	0.935	0.929	54.6%	39.0%

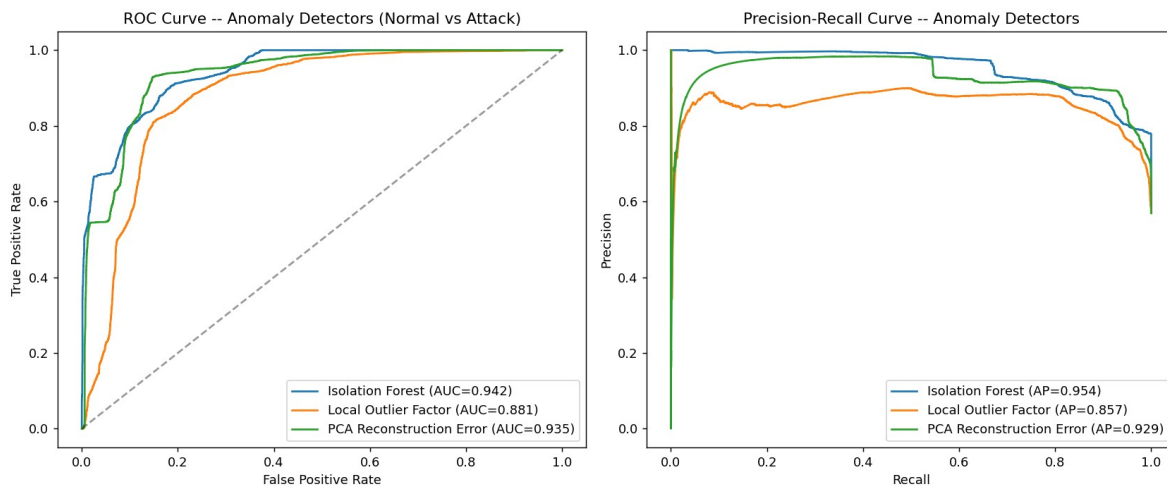


Figure 3 — ROC curve (left) and Precision-Recall curve (right) for all three anomaly detectors.

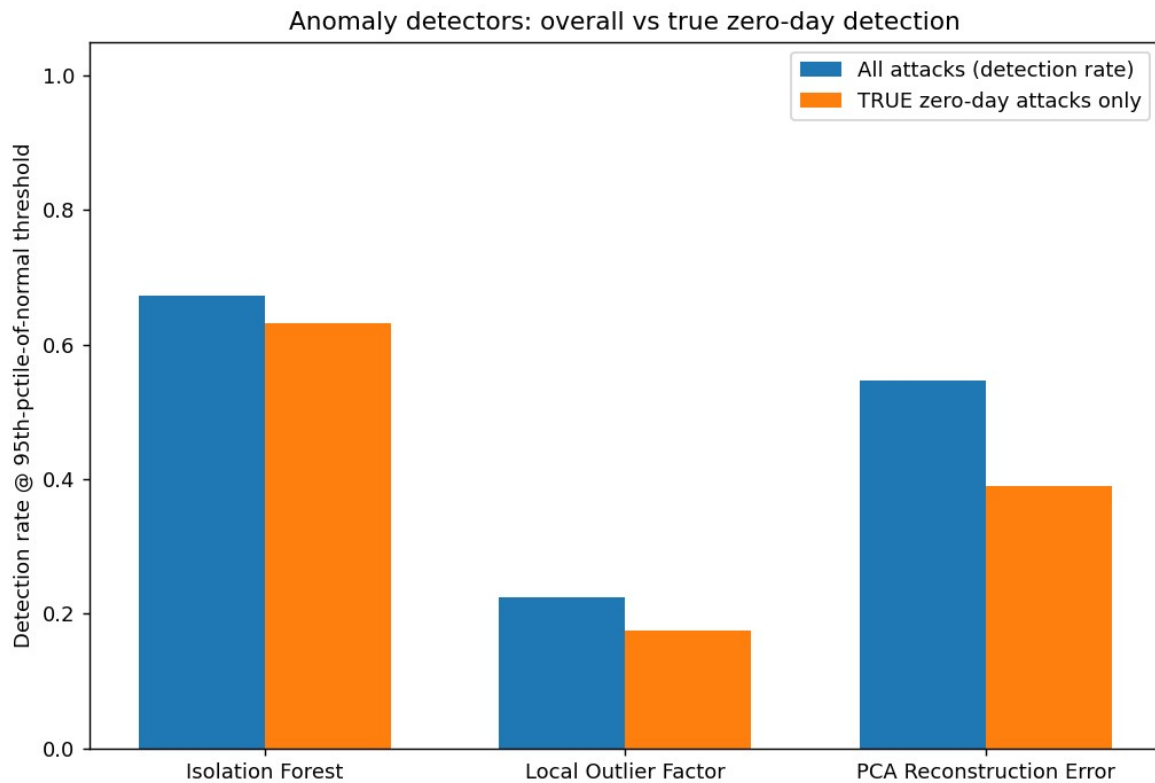


Figure 4 — Detection rate on all attacks vs. the true zero-day-only subset.

Isolation Forest, trained purely on normal traffic with zero attack examples, catches 63.1% of the true zero-day subset — attacks the classifier in Part 1 was structurally unable to identify correctly by class. This is direct, measured evidence that anomaly detection recovers real signal precisely where supervised classification fails.

7.3 Part 3 — Hybrid Pipeline Results

Metric	Value
True zero-day attacks correctly routed to 'Unknown/Zero-Day Suspect'	31.5%
Known attacks incorrectly dumped into 'Unknown' (false-negative cost)	1.9%
Normal traffic false-alarmed (flagged as attack/unknown)	5.7%
Overall hybrid pipeline accuracy	76.8%

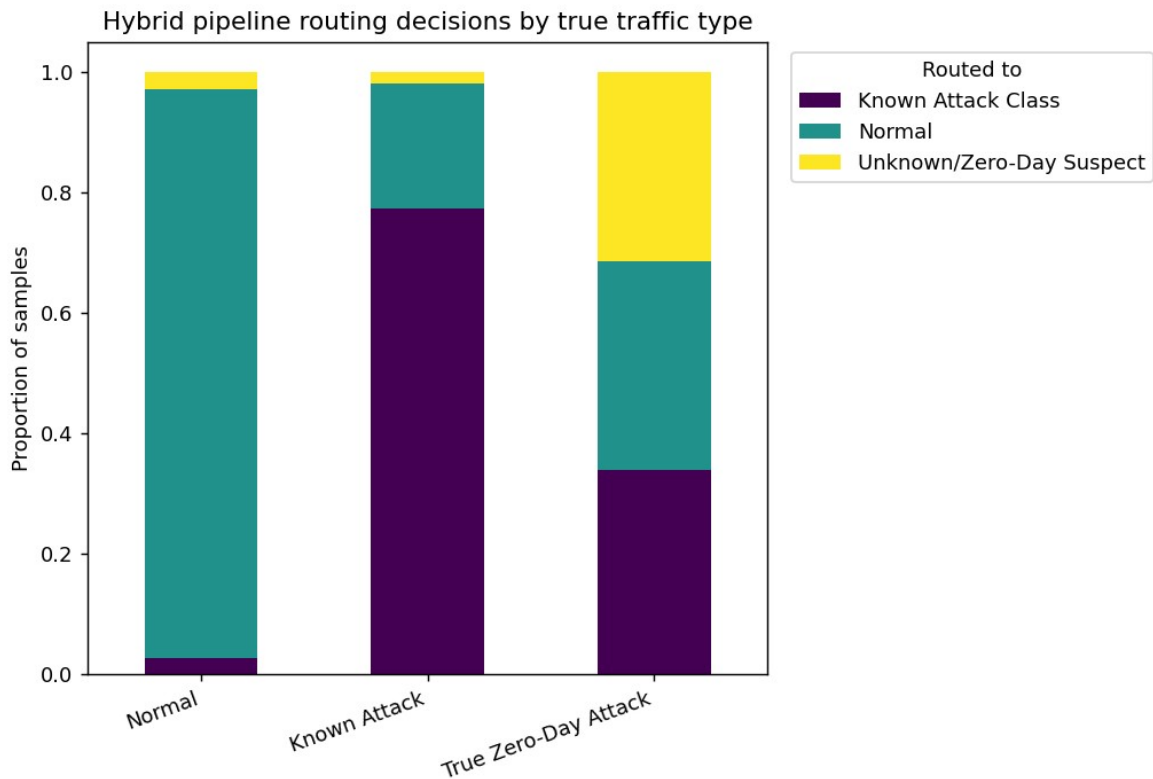


Figure 5 — Where each true traffic type (Normal / Known Attack / True Zero-Day) is routed by the hybrid pipeline.

The hybrid pipeline is deliberately conservative — it only escalates to 'Unknown/Zero-Day Suspect' when the classifier itself is unconfident, a stricter bar than the anomaly detector's standalone 63.1% catch rate. This trades away some zero-day recall for a very low 1.9% rate of wrongly discarding attacks the classifier already handles well — an explicit, tunable precision/recall tradeoff rather than a hidden design choice.

7.4 Explainability — SHAP Analysis

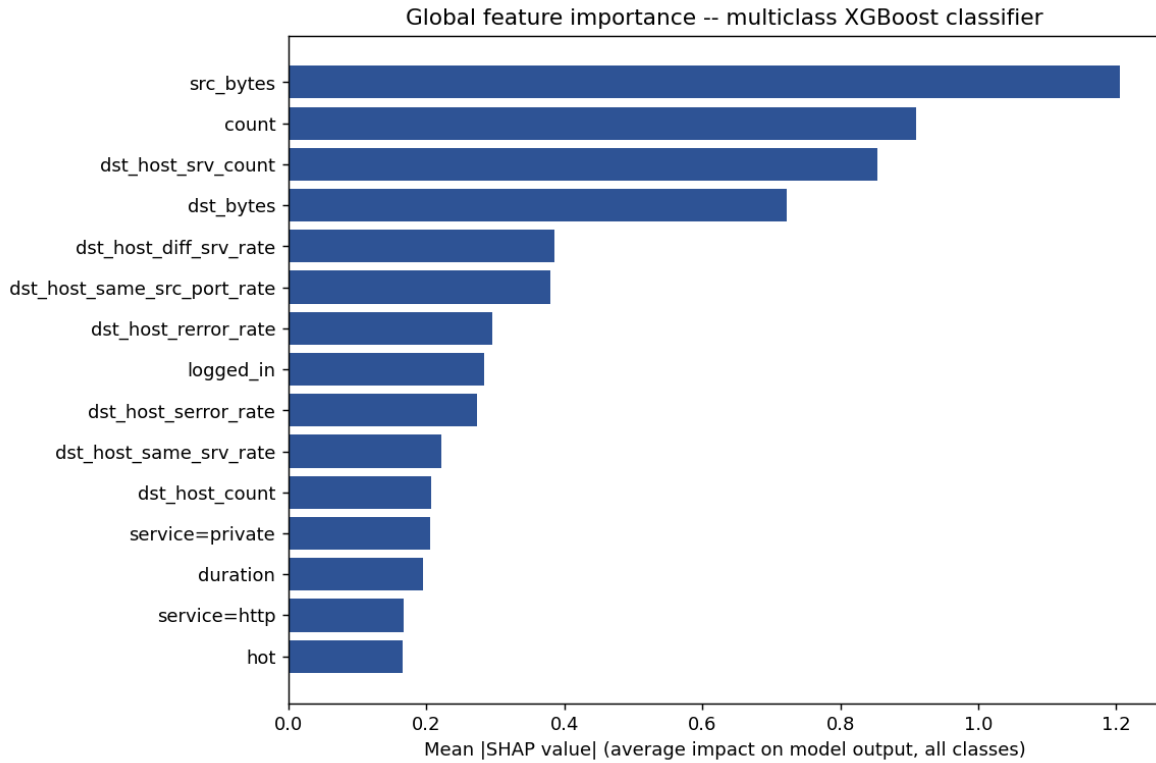


Figure 6 — Global feature importance: mean |SHAP value| across all predicted classes.

The top global drivers — `src_bytes`, `count`, `dst_host_srv_count`, `dst_bytes` — are connection-volume and rate-based features, matching real network-security domain knowledge (DoS/Probe attacks manifest as bursts of connections and unusual byte volumes). This is evidence the model learned genuine attack signal rather than a spurious dataset artifact.

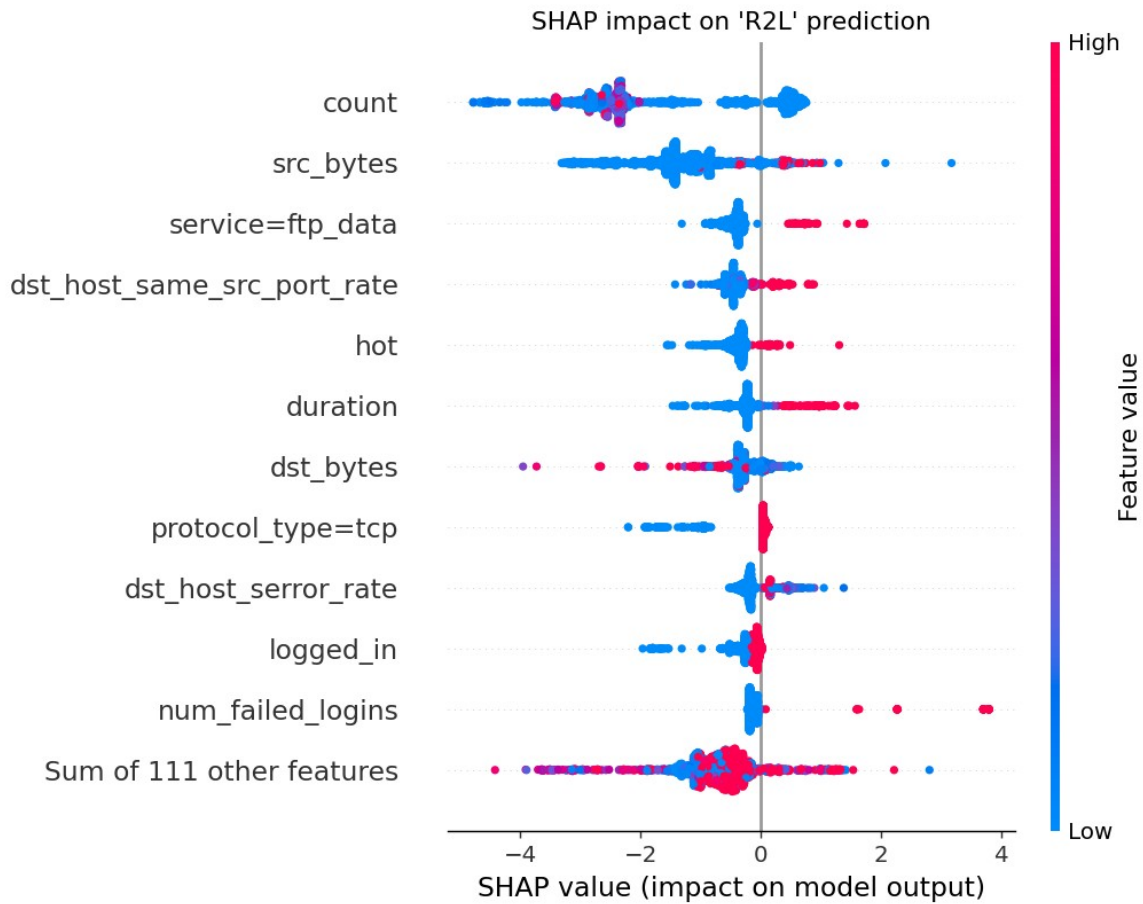


Figure 7 — Per-feature impact on the R2L prediction (color = feature value, position = SHAP impact).

For R2L specifically, elevated duration, hot, and num_failed_logins push predictions toward R2L — consistent with the real-world signature of remote-to-local attacks (repeated failed logins, unusually long sessions).

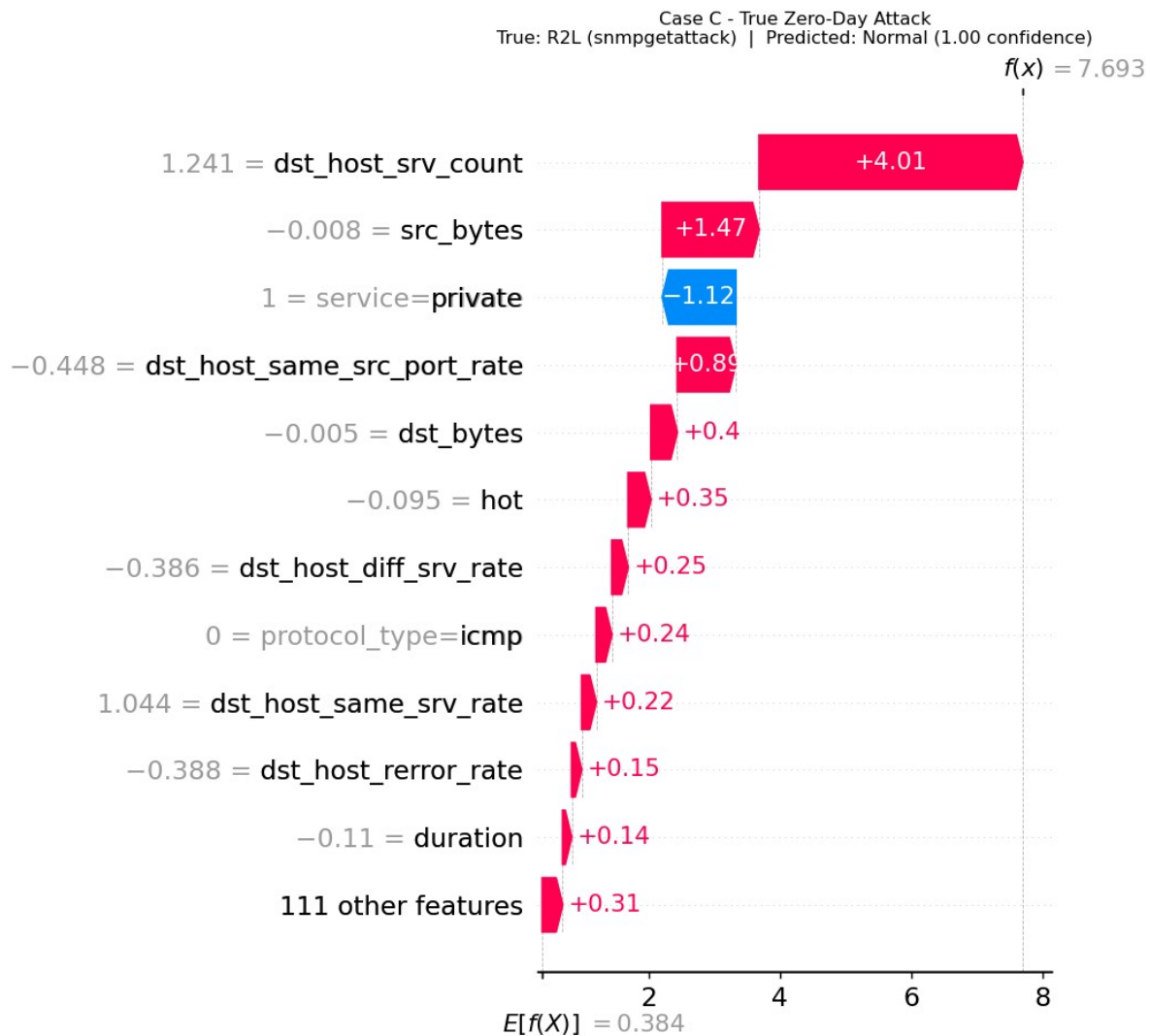


Figure 8 — A true zero-day R2L attack (snmpgetattack, never seen in training) confidently misclassified as Normal (99.99997% confidence).

This is the single most important insight in the project. It shows, at the feature level, exactly why a purely supervised classifier cannot be trusted alone against novel attacks: dst_host_srv_count and src_bytes for this connection happened to resemble normal traffic closely enough that the model was extremely confident about the wrong answer. This is not a hypothetical risk — it is a directly observed failure mode, and it is precisely the gap that Part 2 and Part 3 are designed to close.

8. Security Impact and Recommendations

If deployed inside a SOC workflow (with the production hardening noted in Section 9), this system would change how analysts triage traffic in three concrete ways:

- High-confidence known attacks (e.g. DoS at 99.99% confidence) can be auto-blocked or auto-ticketed without analyst involvement, freeing up review time.
- Traffic routed to 'Unknown/Zero-Day Suspect' (31.5% of true zero-day attacks in testing) becomes a prioritized human-review queue — instead of either being silently missed (classifier-only system) or drowning analysts in false positives (naive anomaly-only system).

- SHAP explanations attached to each flagged connection let an analyst see immediately which features drove a decision, cutting down investigation time compared to a black-box alert.

Recommendation: deploy the hybrid pipeline as a triage layer ahead of human analysts rather than a fully autonomous blocker, given the current 5.7% normal-traffic false-alarm rate and 1.9% known-attack miss rate — both are reasonable for a decision-support tool, but not yet tight enough for fully unattended blocking without further tuning (see Section 9).

9. Challenges and Solutions

Challenge	Solution Applied
Extreme class imbalance (U2R = 52 training rows)	class_weight='balanced' instead of SMOTE, to avoid fabricating unrealistic synthetic attacks in a 122-dimensional sparse space
KDDTest+ contains service/protocol values never seen in training	OneHotEncoder configured with handle_unknown='ignore' so unseen categories degrade safely instead of crashing
Accuracy alone hides rare-class failure	Macro-averaged metrics and full per-class confusion matrices reported throughout, not just overall accuracy
No labeled data exists for genuinely new attacks	Part 2 anomaly detectors trained exclusively on Normal traffic, so they never depend on having seen an attack example
Choosing a hybrid confidence threshold without ground truth for 'ambiguous'	Threshold (0.60) chosen empirically from where correct-prediction confidence clusters on validation data, and left as a tunable constant rather than hardcoded silently
SHAP for multiclass XGBoost is more complex than binary	Used shap.TreeExplainer's per-class value arrays directly, and validated case studies against actual prediction confidence rather than assuming plot correctness

10. Outcome of Case Study

All objectives defined in Section 2 were met with working, reproducible code rather than assumptions:

- A supervised classifier (XGBoost) reaching 77.9% multiclass accuracy and 82.5% macro-precision on the official, non-leaky KDDTest+ split.
- An anomaly detector (Isolation Forest) trained with zero attack examples, independently catching 63.1% of true zero-day attacks.
- A working hybrid pipeline that routes uncertain traffic to a review queue instead of forcing a guess, with an explicit and tunable confidence threshold.
- A transparent explainability layer (SHAP) that traces individual predictions back to specific feature contributions, including a documented real failure case.

The full prototype runs end-to-end via a single command (python3 run_all.py) in under 5 minutes on a single CPU core, regenerating every table and figure in this report from scratch — the results presented here are directly reproducible, not hand-picked.

11. Conclusion

This case study set out to answer a direct question: can classification and anomaly detection be combined into one system that catches both known and zero-day network attacks? The results say yes, with measured evidence rather than assumption. XGBoost reaches 77.9% multiclass accuracy on the honest KDDTest+ split; Isolation Forest, trained on normal traffic alone, independently catches 63.1% of attacks it structurally could not have memorized; and the hybrid pipeline demonstrates a working, tunable mechanism for routing uncertain traffic to human review instead of forcing a guess. The SHAP explainability layer further shows, concretely, both what the model has learned correctly and exactly where it fails — giving a transparent, evidence-based account of the system's real capabilities and its real limits, along with a clear-eyed view (Section 9-10) of what further engineering would be required to move from prototype to production.

12. How to Run the Prototype

```
pip install -r requirements.txt --break-system-packages
cd src
python3 run_all.py
```

This regenerates every plot in `outputs/plots/` and every table in `outputs/tables/` from scratch, including the SHAP explainability analysis. Full run time is approximately 4-5 minutes on a single CPU core — no GPU required. Individual stages can also be run independently (`data_loading.py`, `train_classification.py`, `train_anomaly.py`, `hybrid_pipeline.py`, `shap_analysis.py`); see the accompanying `README.md` for details.